

Capitolul 1

Introducere în MATLAB

MATLAB este un mediu de programare de uz general, ideal pentru implementarea unor metode matematice și numerice. Este folosit ca un calculator de mare performanță pentru probleme mici și ca un limbaj de programare cu toate caracteristicile specifice, pentru probleme mari. O calitate importantă a MATLAB-ului este dată de bibliotecile de funcții, care pot face calcule complicate scurt, precis, și ușor de scris în cod de nivel înalt.

Acest laborator conține o scurtă introducere în comenzile și caracteristicile MATLAB-ului. Detalii pot fi găsite în Help-ul MATLAB-ului, în MATLAB User's Guide, și pe site-ul web <https://www.mathworks.com/help/matlab/>.

1.1 Pornirea MATLAB-ului

MATLAB este pornit dând dublu click pe iconița specifică acestuia. Închiderea MATLAB-ului se poate face închizând fereastra acestuia, din meniul File/Exit, sau tastând

```
>> exit
```

în linia de comandă care apare după deschidere.

Tastați comanda

```
>> a=5
```

urmată de Enter. MATLAB va da aceeași informație care a fost introdusă. Introduceți, de asemenea, și comenzile

```
>> b=3
>> c=a+b
>> c=a*b
>> d=log(c)
>> who
```

pentru a vă obișnui cu modul de funcționare al MATLAB-ului. Dacă se introduce ; după o comandă, nu se va mai afișa rezultatul acelei comenzi. Comanda **who** returnează o listă a tuturor variabilelor pe care le-am definit.

MATLAB are un help online foarte detaliat. Pentru a obține informații despre o anumită comandă, introduceți comanda **help** urmată de numele comenzii respective. De exemplu,

```
>> help log
```

furnizează informații despre comanda `log`. De asemenea, MATLAB are un meniu Help care conține descrierea și exemple de folosire pentru toate comenzile.

Pentru a șterge variabila `a`, introduceți

```
>> clear a
```

Dacă se introduce

```
>> clear
```

se vor șterge toate variabilele definite anterior. Pentru a repeta o anumită comandă, folosiți săgeata „în sus”.

MATLAB efectuează în mod normal toate calculele în dublă precizie IEEE, ceea ce înseamnă aproximativ cu 16 zecimale exacte. Formatul de afișare numerică poate fi schimbat folosind comanda `format`. Introducând `format long` va schimba formatul în care numerele sunt afișate. De exemplu, numărul $1/3$ va fi afișat diferit depinzând de formatul curent:

```
>> format short e
>> 1/3
3.3333E-001
>> format long
>> 1/3
0.33333333333333
>> format long e
>> 1/3
3.33333333333333E-001
>> format bank
>> 1/3
0.33
>> format hex
>> 1/3
3fd5555555555555
>> format short
>> 1/3
0.3333
```

Mai mult control asupra formatului ieșirii este dat de comanda `fprintf`. Comenzile

```
>> x=0:0.1:1;
>> y=x.^2;
>> fprintf('%8.5f %8.5f \n',[x;y])
```

vor afișa tabelul

```
0.00000 0.00000
0.10000 0.01000
0.20000 0.04000
0.30000 0.09000
0.40000 0.16000
0.50000 0.25000
0.60000 0.36000
0.70000 0.49000
0.80000 0.64000
0.90000 0.81000
1.00000 1.00000
```

1.2 Grafice de funcții

Pentru a realiza grafice de funcții, exprimăm datele ca vectori în direcțiile X și Y. De exemplu, comenzile

```
>> a=[0.0 0.4 0.8 1.2 1.6 2.0];
>> b=sin(a);
>> plot(a,b)
```

vor trasa o aproximare liniară pe porțiuni a graficului funcției $y = \sin x$ pe $0 \leq x \leq 2$. În acest caz, **a** și **b** sunt vectori linie de dimensiune 6. Un mod mai compact de a defini vectorul **a** este de a folosi comanda

```
>> a=0:0.4:2;
```

Această comandă definește **a** ca fiind un vector linie ale cărui intrări încep la 0, se incrementează cu 0.4, și se termină la 2, care este identic cu cel din definiția, mai lungă, de mai sus. O versiune mai precisă a întregii perioade a curbei sinus rezultă în urma comenzilor:

```
>> a=0:0.02:2*pi;
>> b=sin(a);
>> plot(a,b)
```

Pentru a trasa graficul funcției $y = x^2$ pe $0 \leq x \leq 2$, putem folosi comenzile:

```
>> a=0:0.02:2;
>> b=a.^2;
>> plot(a,b)
```

Caracterul “.” care precedă operatorul de ridicare la putere poate părea ciudat. El determină operatorul de ridicare la putere să fie vectorizat, mai precis, de a ridica la pătrat fiecare intrare a vectorului **a**. După cum vom vedea în secțiunea următoare, MATLAB tratează orice variabilă ca o matrice. Omiterea punctului în această situație va însemna înmulțirea matricii de dimensiune 1×101 cu ea însăși, folosind regulile înmulțirii de matrici, ceea ce este imposibil. Dacă încercăm această operație, MATLAB va returna o eroare. În general, MATLAB interpretează o operație precedată de punct ca însemnând faptul că operația trebuie aplicată pe fiecare intrare a matricii, și nu ca o operație între matrici.

Există și tehnici mai avansate de trasare a graficelor.

Putem folosi comanda `plot(x1,y1,x2,y2,x3,y3)` pentru a trasa trei curbe în același grafic, unde **xi**, **yi** sunt perechi de vectori de aceeași lungime. Introduceți `help plot` pentru a vedea diferitele tipuri de linii solide, punctate, sau întrerupte și diferitele tipuri de simboluri (cercuri, puncte, triunghiuri, pătrate, etc.) care pot fi folosite pentru grafice.

Comanda `subplot` împarte fereastra în care se vor trasa graficele în mai multe părți. Declarația `subplot(abc)` împarte fereastra într-o grilă de dimensiune $a \times b$ și folosește căsuța *c* pentru a trasa graficul. De exemplu,

```
>> subplot(121),plot(x,y)
>> subplot(122),plot(x,z)
```

trasează primul grafic în stânga ferestrei, iar pe al doilea în dreapta ferestrei. Comanda `figure` deschide o fereastră nouă pentru grafice și comută între ferestrele deschise, dacă este nevoie să se vizualizeze mai multe grafice în același timp.

Graficele unor suprafețe tridimensionale sunt trasate folosind comanda `mesh`. De exemplu, funcția $z = \sin(x^2 + y^2)$ pe domeniul $[-1, 1] \times [-2, 2]$ poate fi reprezentată grafic astfel:

```
>> [x,y]=meshgrid(-1:0.1:1,-2:0.1:2);
>> z=sin(x.^2+y.^2);
>> mesh(x,y,z)
```

Vectorul `x` creat de comanda `meshgrid` este format din 41 de rânduri egale cu vectorul linie `-1:0.1:1` de dimensiune 21, și similar, `y` este format din 41 de rânduri ale vectorului linie `-2:0.1:2` de dimensiune 21. Înlocuind `mesh` cu `surf`, se trasează o suprafață colorată peste grilă.

1.3 Programarea în MATLAB

Rezultate mai sofisticate pot fi obținute scriind programe în MATLAB. Un fișier script este un fișier care conține o listă de comenzi MATLAB. Fișierul script are sufixul `.m`, de unde vine denumirea de m-fișier dată acestor fișiere. De exemplu, se poate folosi editorul MATLAB, pentru a crea fișierul `cubrt.m`, care conține următoarele linii:

```
y=1;
n=15;
z=input('Enter z:');
for i = 1:n
    y = 2*y/3 + z/(3*y^2)
end
```

Pentru a rula acest program, introduceți comanda `cubrt` în linia de comandă MATLAB. Motivul pentru care acest cod converge la rădăcina cubică va deveni evident din studiul metodei Newton în Capitolul 2. Observăm că linia care definește noua iterație a lui `y` nu se termină cu `;`. Aceasta ne permite să vedem progresia aproximărilor, pe măsură ce acestea se apropie de rădăcina cubică.

Folosind abilitățile grafice ale MATLAB-ului, putem analiza datele din algoritmul de determinare a rădăcinii cubice. Să considerăm programul `cubrt1.m`:

```
y(1)=1;
n=15;
z=input('Enter z:');
for i = 1:n-1
    y(i+1) = 2*y(i)/3 + z/(3*y(i)^2);
end
plot(1:n,y)
title('Iterative method for cube roots')
xlabel('Iteration number')
ylabel('Approximate cube root')
```

Rulați programul de mai sus cu `z = 64`. După terminarea acestuia, introduceți comenzile

```
>> e=y-4;
>> plot(1:n,e)
```

Prima comandă scade rădăcina cubică corectă 4 din fiecare intrare a vectorului `y`. Această diferență reprezintă eroarea `e` la fiecare pas al iterației. A doua comandă trasează grafic eroarea.

Crearea unui fișier script pentru codul MATLAB este preferabilă dacă acesta este mai lung de câteva linii. Un fișier script poate apela alte fișiere script, inclusiv pe el însuși.

Un proces MATLAB în desfășurare poate fi oprit folosind combinația de taste **Ctrl+C**.

1.4 Controlul fluxului

Bucloa `for` a fost introdusă în programul anterior de găsim a rădăcinii cubice. MATLAB dispune de mai multe comenzi pentru controlul fluxului într-un program. Mare parte dintre acestea, inclusiv bucloa `while` și instrucțiunile `if` și `break`, sunt familiare pentru oricine cunoaște un limbaj de programare de nivel înalt. De exemplu,

```
n=5;
for i=1:n
    for j=1:n
        a(i,j)=1/(i+j-1);
    end
end
a
```

crează și afișează o matrice Hilbert de dimensiune 5×5 . Rezultatele parțiale nu sunt afișate datorită faptului că atribuirea se încheie cu `;`, iar comanda `a` de la sfârșit afișează rezultatul final. Observăm că fiecare `for` trebuie să fie terminat printr-un `end`. Este o idee bună, deși nu este obligatoriu în MATLAB, de a identifica buclele pentru a putea fi citite mai ușor.

Comanda `while` funcționează asemănător:

```
n=5;
i=1;
while i<=n
    j=1;
    while j<=n
        a(i,j)=1/(i+j-1);
        j=j+1;
    end
    i=i+1;
end
a
```

Acest cod produce același rezultat ca bucloa dublă `for` de mai sus.

Instrucțiunea `if` este folosită pentru structuri decizionale, iar comanda `break` permite un salt de ieșire din buclă. Ambele sunt ilustrate mai jos:

```
while 1
    n=input('Enter n, negative number to quit:')
    if n<=0
        break
    end
    r=rem(n,4)
    if r==0
        y=0
    elseif r==1
        y=1
    elseif r==2
        y=0
    else
        y=-1
    end
end
```

```

    end
    y
end

```

Funcția `rem` calculează restul împărțirii primului parametru la al doilea. Operatorii logici `&` și `|` corespund operațiilor ȘI, respectiv SAU. Comanda `error` oprește execuția unui m-fișier și raportează utilizatorului informația primită ca parametru.

1.5 Funcții

Pe lângă funcțiile de bibliotecă predefinite ca `sin` sau `exp`, MATLAB permite definirea de către utilizator a unor funcții. Comanda

```
>> f=@(x) exp(sin(2*x))
```

crează o funcție care are intrarea x și ieșirea $f(x) = e^{\sin 2x}$. După definirea lui `f` ca mai sus, comanda

```
>> f(0)
```

returnează rezultatul corect $e^{\sin 2(0)} = 1$. Mai mult, definirea cu `@` asignează un function handle lui `f` care poate fi transmis unei alte funcții ca parametru. Dacă definim o altă funcție

```
>> firstderiv=@(f,x,h) (f(x+h)-f(x-h))/(2*h)
```

cu trei intrări `f`, `x`, `h`, comanda

```
>> firstderiv(f,0,0.0001)
```

returnează o aproximare a derivatei acestei funcții în 0. Am folosit function handle-ul `f` al funcției definite de utilizator ca parametru de intrare al celeilalte funcții MATLAB definite de utilizator, `firstderiv`.

O funcție MATLAB poate avea mai multe intrări și mai multe ieșiri. Un exemplu de funcție cu valori vectoriale de mai multe variabile având trei intrări și trei ieșiri este următoarea funcție, care convertește coordonatele rectangulare în coordonate sferice:

```
>> rec2sph=@(x,y,z) [sqrt(x^2+y^2+z^2) acos(z/sqrt(x^2+y^2+z^2)) atan2(y,x)]
```

Această metodă de a defini funcții este utilă atunci când funcția poate fi definită într-o singură linie. Pentru exemple mai complicate, MATLAB permite o a doua metodă de a defini o funcție, folosind un m-fișier special. Sintaxa primei linii trebuie să fie respectată, ca în următorul exemplu, în care numele fișierului este `cubrtf.m`:

```

function y=cubrtf(x)
y=1;
n=15;
for i = 1:n
    y = 2*y/3 + x/(3*y^2)
end

```

Am transferat versiunea de fișier script a programului care aproximează rădăcina cubică într-o funcție MATLAB. Funcția poate fi evaluată astfel:

```
>> c=cubrtf(8)
```

Observăm că o funcție MATLAB diferă de un m-fișier script la prima linie. Numele fișierului, fără sufixul `.m`, trebuie să fie același cu numele funcției, așa cum apare în prima linie. Variabilele într-un fișier de funcție sunt locale în mod implicit, dar pot fi făcute globale folosind comanda `global`.

Combinând cele două abordări de mai sus, o funcție MATLAB anterior definită, de exemplu într-un m-fișier, poate primi un function handle prefixând numele funcției cu semnul `@`. Function handle-ul poate fi dat ca parametru unei alte funcții. De exemplu,

```
>> firstderiv(@cubrtf,1,0.0001)
```

returnează aproximarea 0.3333 pentru derivata lui $x^{1/3}$ în $x = 1$.

O funcție mai complicată poate folosi mai multe variabile de intrare și mai multe variabile de ieșire. De exemplu, funcția de mai jos apelează funcțiile MATLAB predefinite `mean` și `std`, și le colectează pe ambele într-un vector:

```
function [m,sigma]=stat(x)
m=mean(x);
sigma=std(x);
```

Acest fișier `stat.m` poate fi apelat introducând `stat(x)`, unde `x` este un vector, și va returna media și deviația standard a intrărilor vectorului `x`.

Comanda `nargin` returnează numărul de parametri de intrare pentru o funcție. Cu această comandă, comportarea funcției se poate schimba, depinzând de cât de mulți parametri sunt dați funcției.

Un exemplu de funcție definită pe ramuri este

$$h(x) = \begin{cases} x + 2, & \text{pentru } x \leq -1 \\ 1, & \text{pentru } -1 < x \leq 0 \\ \cos x, & \text{pentru } x > 0 \end{cases}.$$

Funcția $h(x)$ poate fi reprezentată creând m-fișierul de funcție MATLAB `h.m` cu următorul conținut:

```
function y=h(x)
p1=(x<=-1);
p2=(x>-1).*(x<=0);
p3=(x>0);
y=p1.*(x+2)+p2.*1+p3.*cos(x);
```

Aici, folosim evaluarea booleană a expresiilor condiționale considerând 1 ca fiind `true` și 0 ca fiind `false`. Folosim, de asemenea, punctul care precedă operațiile aritmetice pentru a le vectoriza, permițând astfel ca intrarea `x` să fie un vector de numere. Acum, `h` poate fi dată altor funcții MATLAB ca parametru prin intermediul function handle-ului `@h`. De exemplu,

```
>> ezplot(@h,[-3 3])
```

trasează grafic funcția definită pe ramuri `h`, și

```
>> fzero(@h,1)
```

găsește o rădăcină a lui $h(x)$ în apropierea lui 1. Putem avea încredere în corectitudinea rezultatului apelului de funcție de mai jos?

```
>> firstderiv(@h,-1,0.0001)
```

1.6 Operații cu matrici

Cheia puterii și versatilității MATLAB-ului este dată de gradul de complexitate al structurii de date a variabilelor sale. Fiecare variabilă în MATLAB este o matrice $m \times n$ de numere în virgulă flotantă în dublă precizie. Un scalar este pur și simplu un caz special de matrice de dimensiune 1×1 . Sintaxa

```
>> A=[1,2,3;4,5,6]
```

sau

```
>> A=[1 2 3;4 5 6]
```

definește o matrice **A** de dimensiune 2×3 . Comanda **B=A'** creează o matrice **B** de dimensiune 3×2 care este transpusa lui **A**. Matricile de aceeași dimensiune pot fi adunate și scăzute folosind operatorii **+** și **-**. Comanda **size(A)** returnează dimensiunile matricii **A**, iar **length(A)** returnează cea mai mare dintre cele două dimensiuni ale matricii **A**.

MATLAB oferă mai multe comenzi care permit construirea mai ușoară a matricilor. De exemplu, **zeros(m,n)** returnează o matrice de zerouri de dimensiune $m \times n$. Dacă **A** este o matrice, atunci **zeros(size(A))** produce o matrice de zerouri care are aceeași dimensiune ca **A**. Comenzile **ones(m,n)** (care returnează o matrice de 1-uri) și **eye(m,n)** (care returnează matricea identitate) funcționează în aceeași manieră. De exemplu,

```
>> A=[eye(2) zeros(2,2);zeros(2,2) eye(2)]
```

este o modalitate complicată, dar exactă de a construi o matrice identitate de dimensiune 4×4 .

Operatorul **:** poate fi folosit pentru a extrage o submatrice dintr-o matrice. De exemplu,

```
>> b=A(1:3,2)
```

asignează lui **b** primele trei intrări din cea de-a doua coloană a lui **A**. Comanda

```
>> b=A(:,2)
```

asignează lui **b** cea de-a doua coloană a lui **A**, și

```
>> B=A(:,1:3)
```

asignează lui **B** submatricea formată din primele trei coloane ale lui **A**.

Matricea $m \times n$ **A** și matricea $n \times p$ **B** pot fi înmulțite folosind comanda **C=A*B**. Dacă matricile nu au dimensiuni compatibile, MATLAB nu va face operația, și va returna un mesaj de eroare.